

Arrays level - 0

Remove Duplicates

- ⇒ Sorted, increasing order $\rightarrow [1, 2, 3, 4, 10, 11, 13] \rightarrow \text{arr}[i+1] > \text{arr}[i]$
- ⇒ Sorted, decreasing order $\rightarrow [10, 9, 8, 7, 6, 2, 1] \rightarrow \text{arr}[i+1] < \text{arr}[i]$
- ⇒ Sorted, non-decreasing order $\rightarrow [1, 2, 2, 3, 3, 4, 5] \rightarrow \text{arr}[i+1] \geq \text{arr}[i]$
(can contain duplicates)
- ⇒ In-place $\rightarrow$ don't create new array just modify in original array.

Examples:

① input: $[1, 1, 2]$

output: 2, $[1, 2, -]$
(or) $[1, 2]$ not important

$[1, 2]$ even size is also not important

But first k elements should be unique.

② input: $[0, 0, 1, 1, 2, 2, 3, 4]$

output: $[0, 1, 2, 3, 4, -, -, -]$ (or) $[0, 1, 2, 3, 4]$ & 5 (return no. of unique elements)
↓
Should maintain relative order

Code:

```
function removeDuplicates(arr)
```

```
{
```

```
    let uniqElePtr = 0;  $\rightarrow$  ptr to track where uniq element should be placed
```

```
    for (let i = 0; i < arr.length; i++)
```

```
    {
```

```
        if (arr[i] > arr[uniqElePtr])
```

```
        {
```

```
            uniqElePtr++;  $\rightarrow$  moving the ptr
```

```
            arr[uniqElePtr] = arr[i];  $\rightarrow$  shifting the elements
```

```
        }
```

```
    }
```

```
    return (uniqElePtr + 1);
```

```
}
```

$[1, 1, 2] \rightarrow [1, 2, 2]$
x x v

dry run:

i	uniqElePtr
0 x	0
1 x	1
2 v	2

return (uniqElePtr + 1)

return (1 + 1)

return 2;

Algo: ① use two pointers (one for iteration, another to store index at which we need to shift)

② check if the element is unique or not.

③ move the pointer which is pointing to index

④ Now shift the element.

⑤ iterate ③ and ④ until array ends in a loop

⑥ Now return index + 1

Remove Element

arr = [0, 1, 2, 2, 3, 0, 4, 2] Val = 2

Code:

```
function removeElement(arr, val)
{
    let track = 0;
    for(i=0; i < arr.length; i++)
    {
        if (arr[i] !== val)
        {
            arr[track] = arr[i];
            track++;
        }
    }
    return track;
}
```

dry run:

track	i	arr
0	0	[0, 1, 2, 2, 3, 0, 4, 2]
1	1	[0, 1, 2, 2, 3, 0, 4, 2]
2	2	[0, 1, 2, 2, 3, 0, 4, 2]
3	3	[0, 1, 2, 2, 3, 0, 4, 2]
4	4	[0, 1, 3, 2, 3, 0, 4, 2]
5	5	[0, 1, 3, 0, 3, 0, 4, 2]
6	6	[0, 1, 3, 0, 4, 0, 4, 2]
7	7	[0, 1, 3, 0, 4, 0, 4, 2]

Algo:

- ① 2 pointers approach
- ② Check if the element is not equal to value we need to remove.
- ③ Shift that element to index pointer.
- ④ move index pointer.
- ⑤ Do/iterate ②, ③ and ④ until pop/array ends.
- ⑥ finally return index.

Reverse String

Ex:

S = ["h", "e", "l", "l", "o"]

O/p: S = ["o", "l", "l", "e", "h"]

Algo:

we need to first element with last. then second with second last and so on.

Algo: ① run a half loop ($< n/2$)

② Swap the elements $\rightarrow$ swap(i, n-i-1)

Code:

```
for(i=0; i < Math.floor(arr.length/2); i++)
{
    let temp = arr[i];
    arr[i] = arr[arr.length-1-i];
    arr[arr.length-1-i] = temp;
}
```


Best time to buy And sell Stocks

- Algo:
- ① maintain (or) track two values min value element and max-profit as we are traversing
 - ② check to keep track of min (if (arr[i] < min))
 - ③ check to find max profit (if (arr[i] - min > maxprofit))

arr = [7, 5, 6, 3, 2, 1]

Code:

```
function maxProfit(arr) {  
  let min = arr[0];  
  let maxprofit = 0;  
  for (i = 1; i < arr.length; i++) {  
    if (arr[i] < min)  
    {  
      min = arr[i]  
    }  
    if (arr[i] - min > maxprofit)  
    {  
      maxprofit = arr[i] - min;  
    }  
  }  
  return maxprofit  
}
```

i	min	maxProfit
1	7	0
2	5	0
3	5	1
4	3	1
5	2	1
6	1	1

return →

Merge Sorted Arrays

Ex:
nums1 = [2, 5, 6, 0, 0, 0] → m=3
nums2 = [1, 2, 3] → n=3

Approach 1:

- ① maintain one pointers at m
- ② Now loop through nums2 and copy these elements to nums1 in the place of zeros.
- ③ now sort the nums1 array using .sort method.

```
var merge = function (nums1, m, nums2, n) {  
  let ptr = m; → ①  
  for (let i = 0; i < n; i++) {  
    nums1[ptr] = nums2[i];  
    ptr++;  
  } → ②  
  nums1 = nums1.sort((a, b) => a - b); → ③  
}
```


Approach 2:

- ① make a copy of nums1 as nums1Copy.
nums1 = [1, 2, 3, 0, 0, 0]
nums1Copy = [1, 2, 3, 0, 0, 0] → slice it → [1, 2, 3]
nums2 = [2, 5, 6]
- ② maintain two pointers at index 0 for numsCopy and nums2

nums1Copy = [1, 2, 3]
 ↑
 ptr1

nums2 = [2, 5, 6]
 ↑
 ptr2
- ③ loop from 0 to m+n (as we need to sort in nums1)
- ④ Now check which is smaller and put it in nums1 and move the respective ptr.

What if $ptr1$ goes out of bound?

what if ptr 2 goes out of bound?

```
if ( (nums1Copy[ptr1] < nums2[ptr2]) && (ptr1 < m)
    || (ptr2 >= n)) {
```

← copy nums1 copy into nums1 →

3

else if

← copy nums2 into nums1 →

3

```
let nums1Copy = nums1.slice(0,m); } → ①
```

let ptr1 = 0; } → (2) → (3)
let ptr2 = 0; }

```
for (i = 0; i < m + n; i++) {
```

```
if ((nums1Copy[ptr1] < nums2[ptr2] && ptr1 < m) || ptr2 >= n) {
    nums1[i] = nums1Copy[ptr1];
```

```
ptr1++;
```

```
} else {
```

```
nums1[i] = nums2[ptr2];
```

```
ptr2++;
```

}

}

(4) 2 Corner cases

4. Approach 3:

- ① we need to maintain 2 ptrs.
- ② This time we will loop from mth to 0 in reverse.
- ③ check which is greater and copy into nums1

Cornel cast

- ④ If nums2 go beyond 0 then break loop
- ⑤ If nums1 go beyond 0 copy remaining elements from nums2 to nums1.


```

let ptr1 = m - 1; } ①
let ptr2 = n - 1; }
for (let i = m + n - 1; i >= 0; i--) { } → ②
  if(ptr2 < 0) → ④
  {
    break; → ⑤
  }
  if((ptr1 >= 0) && (nums1[ptr1] > nums2[ptr2])) → ③
  {
    nums1[i] = nums1[ptr1];
    ptr1--;
  }
  else{
    nums1[i] = nums2[ptr2];
    ptr2--;
  }
}

```

move zeroes

Algo:

[0, 1, 0, 3, 12]

- ① We need to maintain two pointers → 1) to track the index at which we want to shift non zero value
- ② check if element is non-zero value → 2) to iterate which
- ③ if true, then copy that value to ptr₁ which is pointing to track index
- ④ After that move the index value to next position.
- ⑤ if false, simply iterate in the loop.
- ⑥ After loop ends we have the value of index (which is tracking)
- ⑦ Now from that index till the end just fill with zero.

```

function moveZeroes(arr) {
  let index = 0; } → ①
  for (let i = 0; i < arr.length; i++) { } → ⑤
    if (arr[i] !== 0) { → ②
      arr[index] = arr[i]; } → ③
      index++; } → ④
    }
  } → ⑥
  for (index; index < arr.length; index++) { } → ⑦
    arr[index] = 0;
  }
}

```

Max Consecutive Ones

- Algo:
- ① maintain 2 pointers to track current count and max count
 - ② iterate through array
 - ③ check if it is equal to 1
 - ④ if true, then increase current count
 - ⑤ As soon as you encounter 0

⑥ update the max count by checking which one is max (current count or max count)

⑦ then make current count to 0.

Corner case: if you did not encounter 0 but current count is greater than max count. ?

⑧ so to cover corner case after the loop check which is greater and return it.

```
let currCount = 0; } → ①
let maxCount = 0; }
for(let i=0; i<arr.length; i++) → ②
{
  if(arr[i] === 1) → ③
  {
    currCount++; → ④
  }
  else → ⑤
  {
    maxCount = Math.max(currCount, maxCount); → ⑥
    currCount = 0; → ⑦
  }
} → ⑧
return Math.max(currCount, maxCount);
```

missing Numbers

Algo: ① find total sum using the sum of n numbers formula $\frac{n \times (n+1)}{2}$

② Then find partial sum by adding all elements in the given array.

③ Then return the difference of Total and partial sum then we can get missing number.

```
function missingNumber(nums)
{
  let n = nums.length; } → ①
  let totalSum = n*(n+1)/2; }
  let partialSum = 0;
  for(let i=0; i<nums.length; i++) → ②
  {
    partialSum = partialSum + nums[i];
  }
  return totalSum - partialSum; } → ③
}
```

Single Number

Algo: approach 1:

- ① maintain a hashmap
- ② iterate through array
- ③ if it is not present in hashmap add to it set its value to 1.
- ④ if it is present just increment
- ⑤ Now, iterate through that hashmap

approach 2:

Note: $\text{num} \oplus 0 = \text{num}$
 $\text{num} \oplus \text{num} = 0$

- ① Just XOR the values of all elements present in array. as $\text{num} \oplus \text{num} = 0$

if value is equal to 1 then
return that element.

function singleNumber(nums)

```
{  
  let hash = {} → ①  
  for(let i=0;i<nums.length;i++) → ②  
  {  
    if(!hash[nums[i]]) → ③  
    {  
      hash[nums[i]] = 1;  
    }  
    else → ④  
    {  
      hash[nums[i]]++;  
    }  
  }  
  for(let i=0;i<nums.length;i++) → ⑤  
  {  
    if(hash[nums[i]]==1)  
    {  
      return nums[i];  
    }  
  }  
}
```

and $\text{num}(\text{xor}) 0 = \text{num}$
only one element left
all other will be elimi-
-nated.

function singleNumber(nums)

```
{  
  let xor = 0;  
  for(let i=0;i<nums.length;i++) → ①  
  {  
    xor = xor^nums[i];  
  }  
  return xor;  
}
```

↓
This is the best approach
to find the single element
present in an array while
other elements appear twice.